

计算机体系结构

12. 多核处理器入门

李建华

计算机与信息学院

The contents of the slides are adapted from CA course of wisc, princeton, mit, berkeley, edinburgh, and eth.
The uses of the slides of this course are for educational purposes only and should be used only in conjunction with the textbook. Derivatives of the slides must acknowledge the copyright notices of this and the originals.

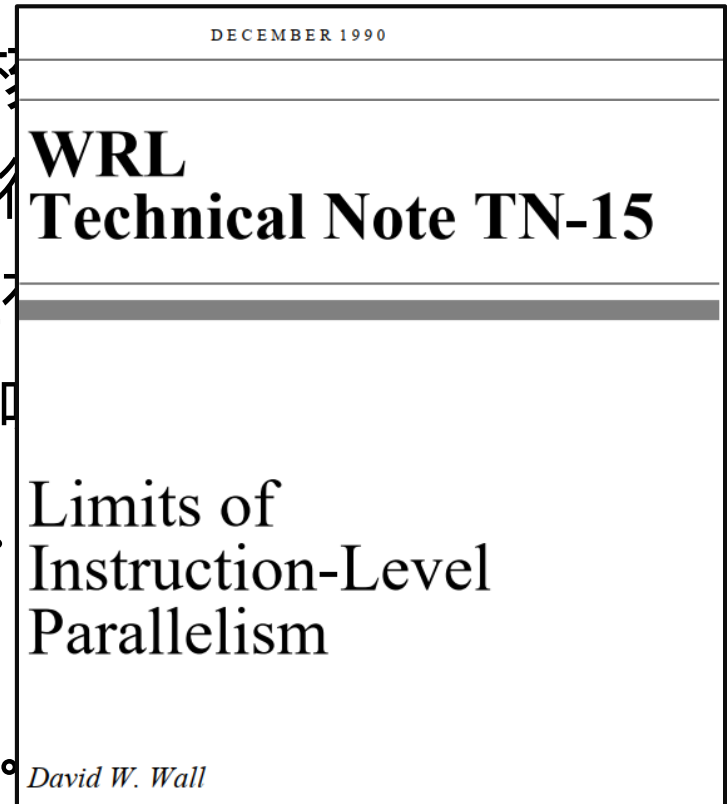
如何进一步提升性能?

- 到这里，我们已经学习了：
 - 流水线及其性能优化技术
 - 存储器及其性能优化技术
- 要进一步提升性能，可行的方法有：
 - 更深的流水线（并不是越深越好）
 - 缩短时钟周期长度（越来越难）
 - 利用其他并行性
 - DLP, TLP, RLP
- 然而，提升性能最直接的方案是：
 - 采用多核处理器 (multicore processor)

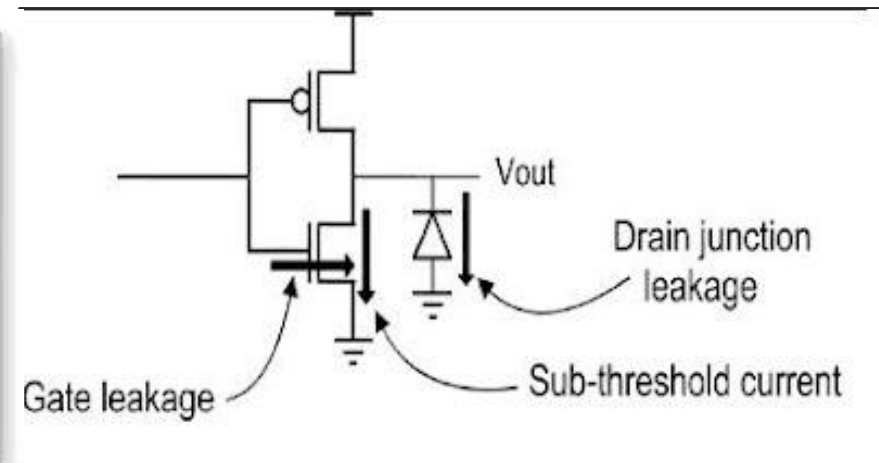
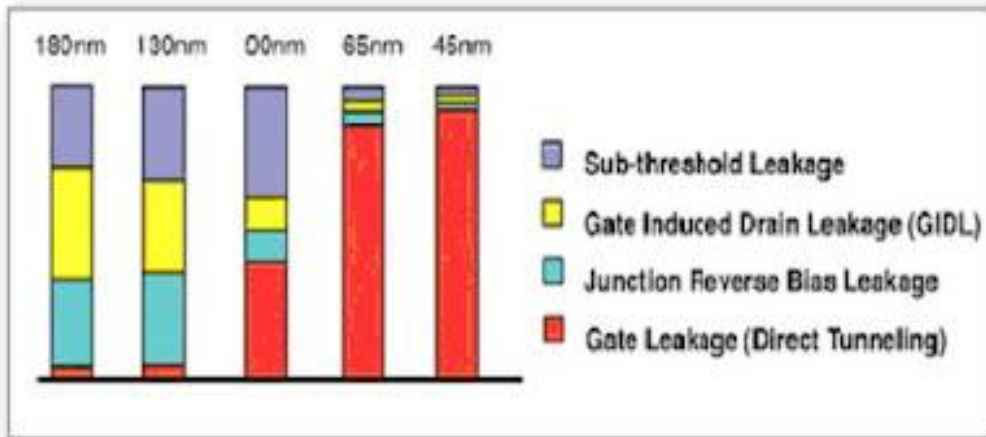
这些内容可自学，或者
研究生阶段再学习。

为什么采用多核处理器？

- 重要原因：摩尔定律支撑多核
 - 摩尔定律是关于晶体管演化路径
 - 更小的特征尺寸 → 单个die上
 - 与此同时，更小的晶体管也意味
- 但是：也要考虑系统的功耗...
 - 果然，后来功耗问题解决不了
 - 从能耗角度，多核是必然选择。
- 另外一个重要原因是继续提升ILP的空间非常有限，或者说程序本身的ILP有限。
- 这里，我们仅仅讨论多核处理器的基础内容。



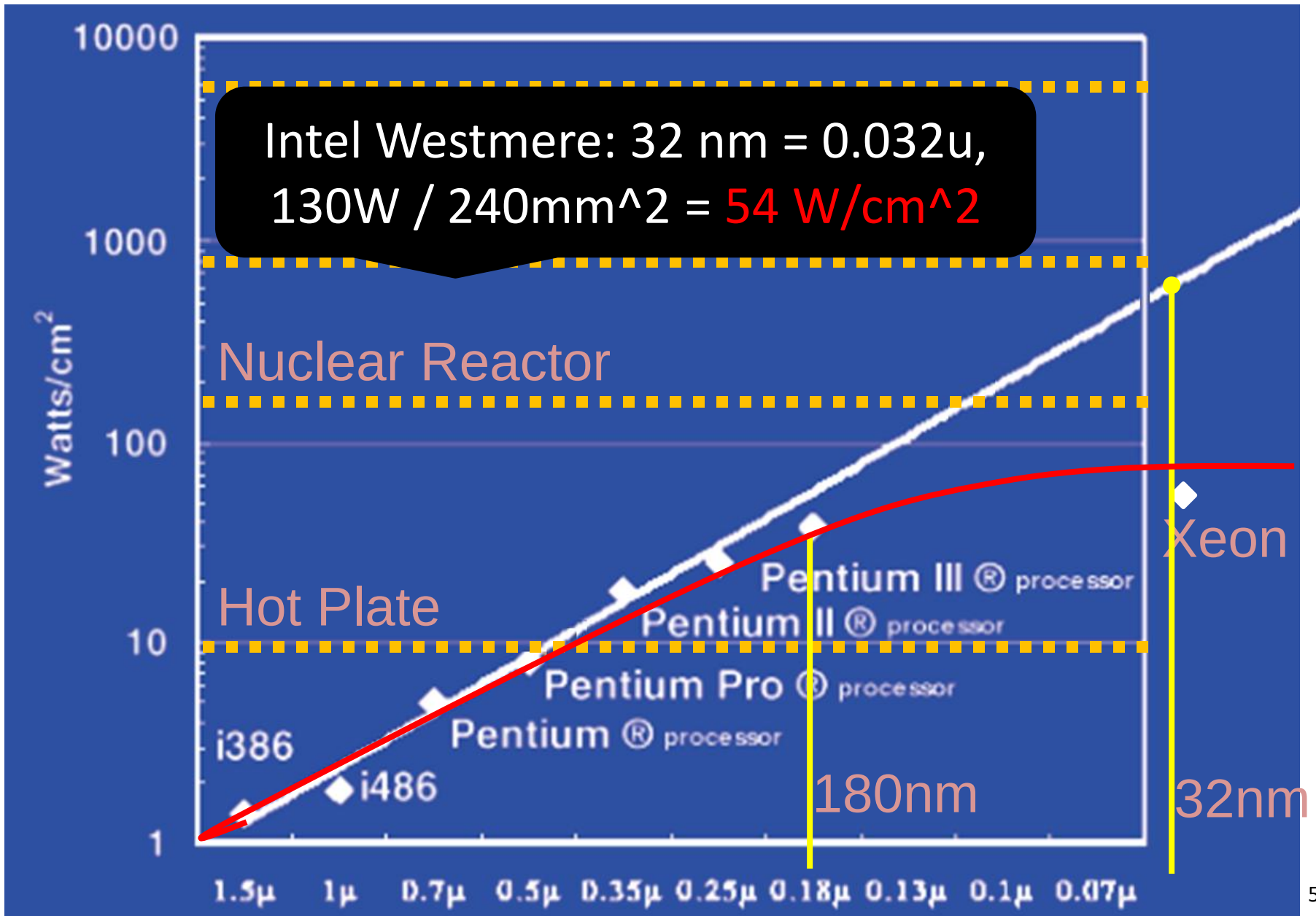
功耗墙 (Power Wall) → Multicore



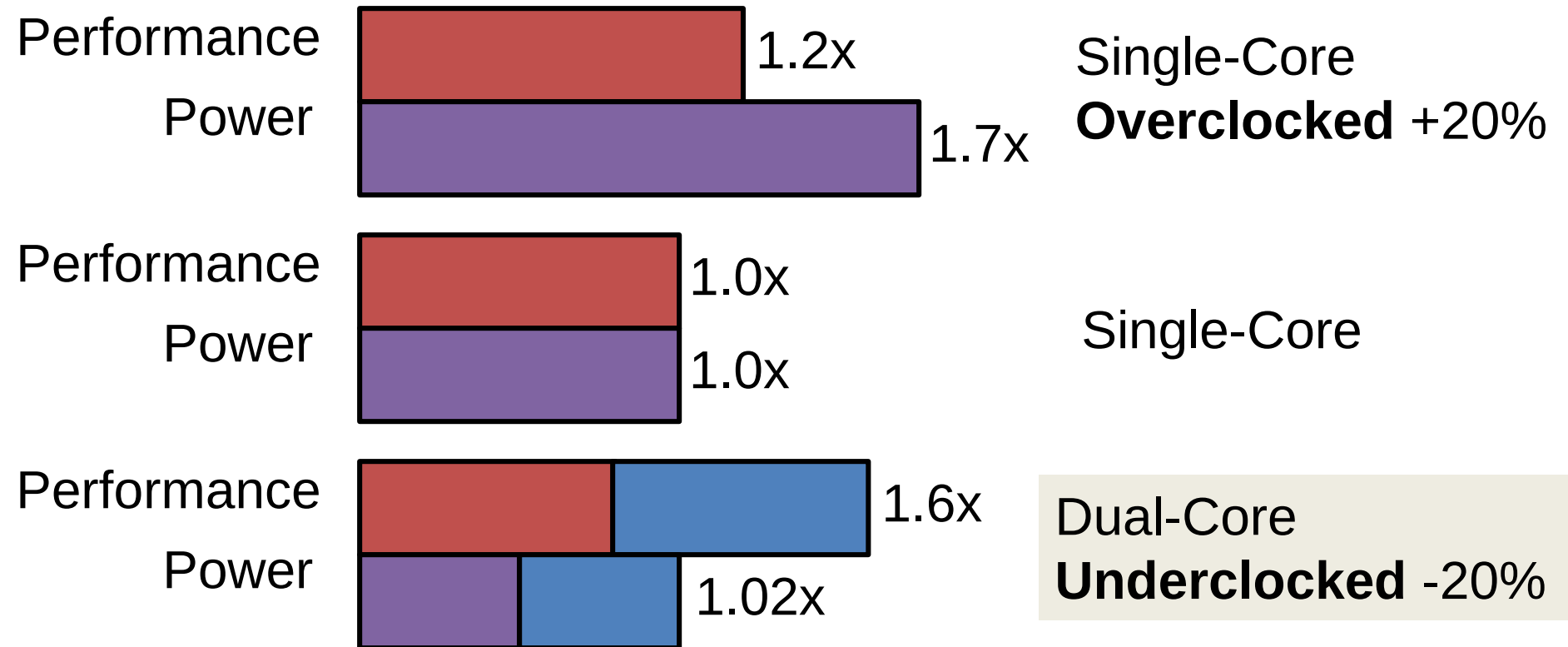
- 那么，功耗墙指什么？
 - 不能进一步降低晶体管的工作电压
 - 泄露功耗 (leakage power ↑) 、稳定性
 - 不能及时去除更多的热量 (more transistor)
 - 受风冷机制的限制

图片来源: <https://asic-soc.blogspot.com/2008/03/leakage-power-trends.html>

功耗问题与制冷散热

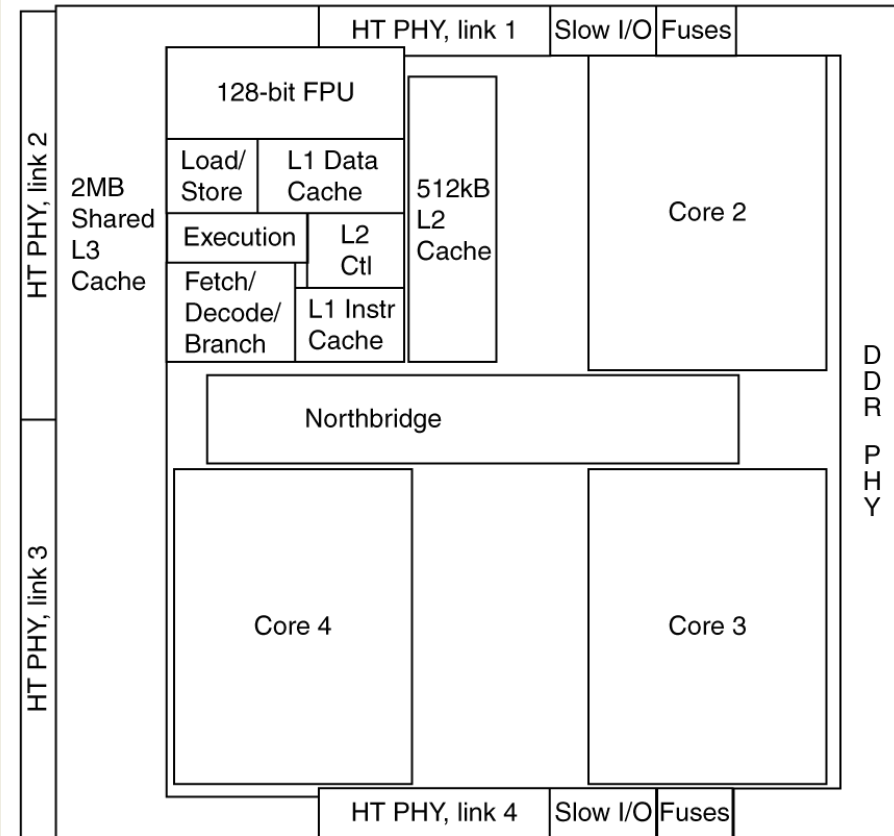
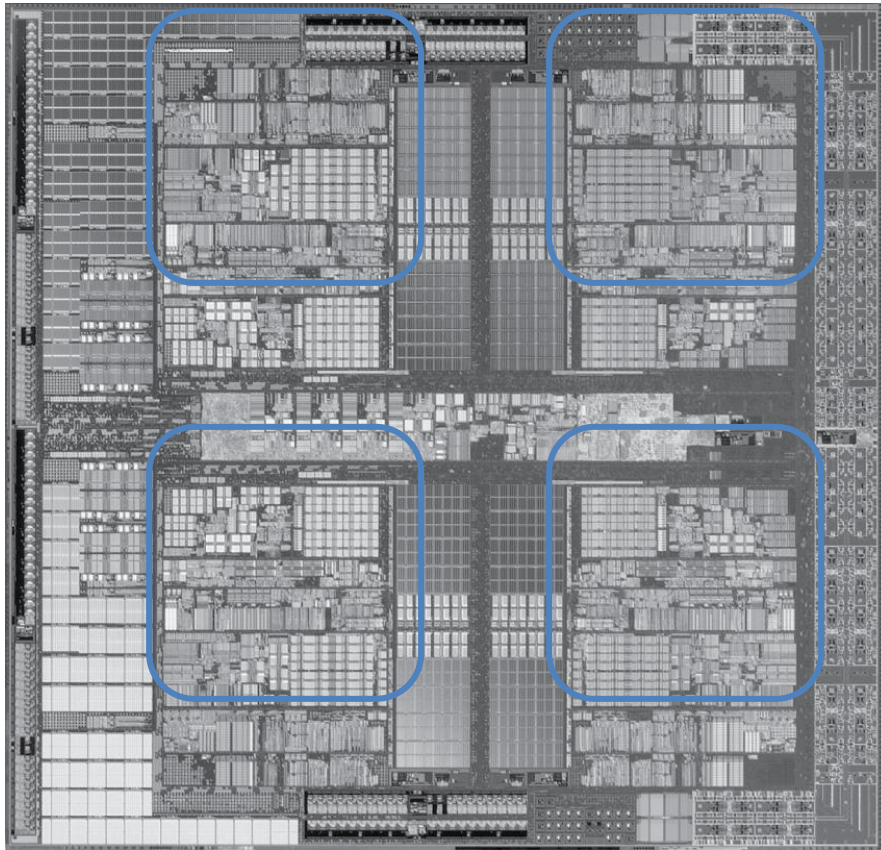


多核处理器的合理性



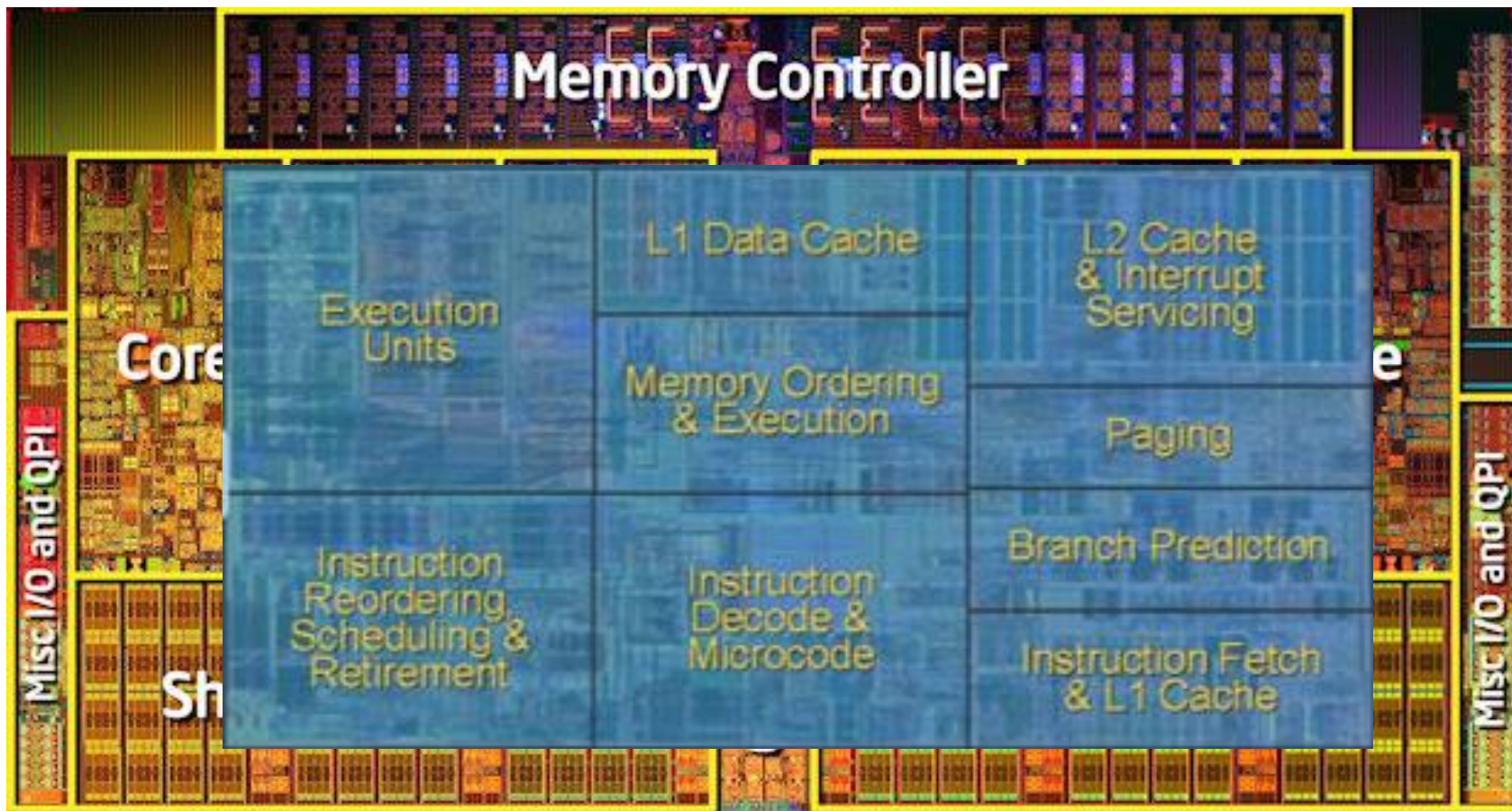
多核处理器内部

- AMD Barcelona Quad-Core: 4 核处理器



多核处理器内部

- Intel Nehalem Hex-Core



提示：这些技术你都掌握了吗？

多核还不够？ + 同时多线程！

- 同时多线程的概念首先由学术界提出？

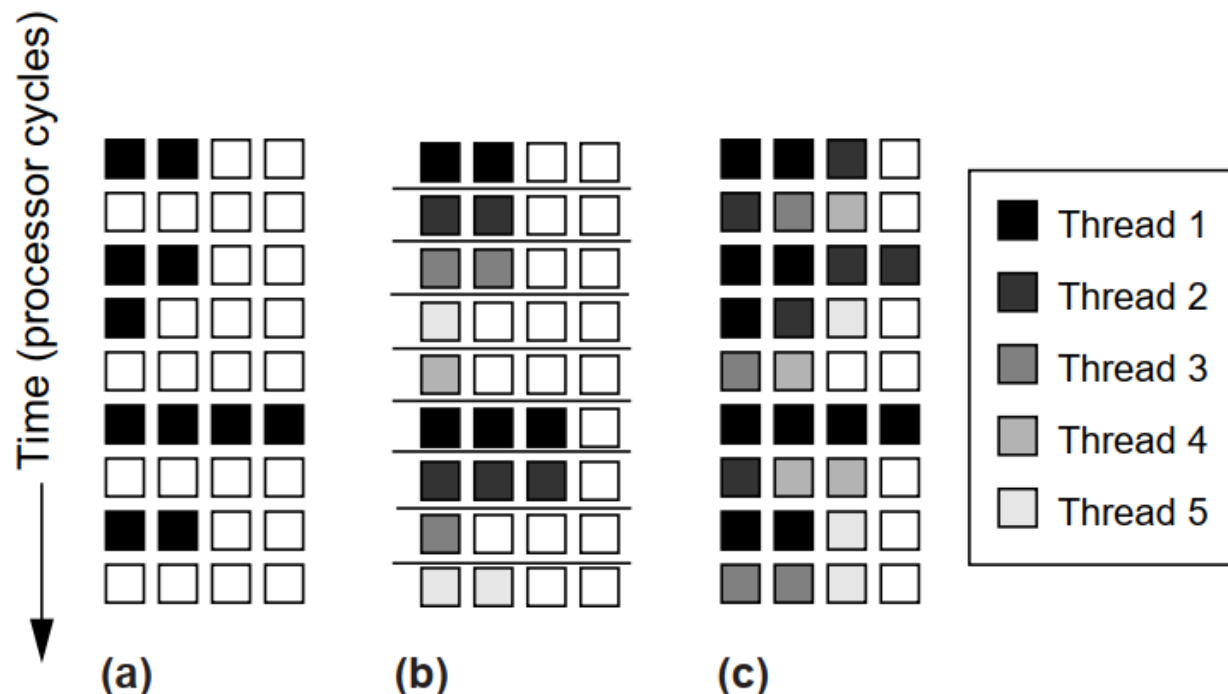
[ieeexplore.ieee.org > document](http://ieeexplore.ieee.org/document/444444) · [翻译此页](#)

Simultaneous multithreading: Maximizing on-chip parallelism ...

Simultaneous multithreading has the potential to achieve 4 times the throughput of a superscalar, and double that of fine-grain multi-threading. ...

作者: DM Tullsen · 1995 · 被引用次数: 2122 · [相关文章](#)

- Hyperthreading (Intel的说法)
 - 让物理上单个核看起来像多个核
- 当发生暂停事件时，在多个硬件线程间切换。
 - 细粒度和粗粒度的问题
 - 普通的多线程，是粗粒度的切换
 - SMT，同时运行多个线程，且可以每个cycle切换线程。
 - 可阅读书本3.12节和第5章相关内容。



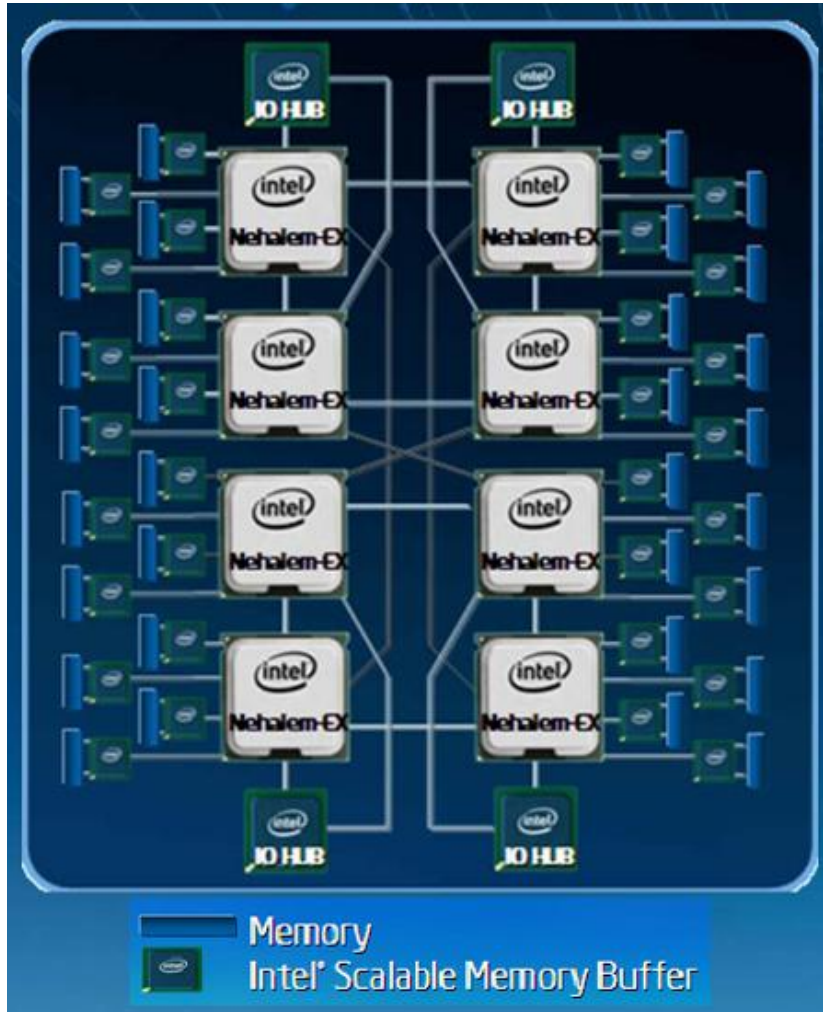
Our SMT model, which can simultaneously execute threads from up to eight hardware contexts, is a straightforward extension of this conventional superscalar. We replicated some superscalar resources to support simultaneous multithreading: state for the hardware contexts (registers and program counters) and per-thread mechanisms for pipeline flushing, instruction retirement, trapping, precise interrupts, and subroutine return. We also added per-thread (address-space) identifiers to the branch target buffer and translation look-aside buffer. Only two components, the instruction fetch unit and the processor pipeline, were redesigned to benefit from SMT's multithread instruction issue.

多核-多发射-超线程

	多核	多发射	超线程
--	----	-----	-----

- Programs: N 1 N
- Num. Pipelines: N 1 1
- Pipeline Width: 1 N N
- 超线程就相当于:
 - HT = 多发射 + 额外的PCs和寄存器 – 相关检测逻辑
 - HT = 多核 – 冗余的功能单元 + 冒险避免机制
- 超线程 (Intel)
 - 让单个核看起来像多个核 (逻辑上)
 - 容易让HT的流水线稳定 + 共享功能单元

多处理器(多核)系统示例



- 8 **multiprocessors**
- 4 core per multiprocessor
- 2 HT per core
- 动态多发射: 4 issue
- 流水线深度: 16
- Note: 每个处理器可以有多个核心。因此，这是一个多处理器系统，多核系统，支持超线程的混合示例。

如何利用多核→并行政程序设计

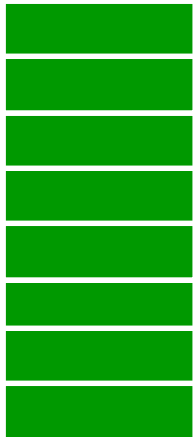
- **问题：**我们有了多核处理器之后，应用程序的性能就可以直接、自然地提升了吗？
- **回答：**并不能，因为软件需要按照并行化的方式去写 (并行计算/并行政程序设计)，对吗？
- 多核处理器面临的困难：
 - 任务划分和负载均衡
 - 协调 和 同步
 - 通信开销
 - 如何写并行政程序？
 - ... 在不知道底层硬件架构细节的情况下？
 - 我们来看一个多线程的[demo](#)！



你写过多线程程序吗？
如果没有，尝试用
pthread/openmp/mpi
写一个？

任务划分 – 理想情况

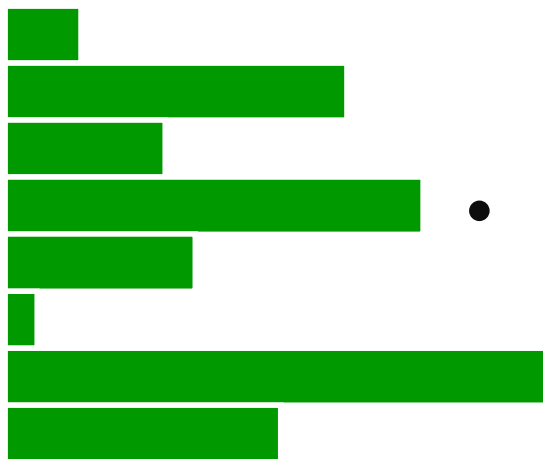
- 进行任务划分，让所有的核芯有事可做。
 - 之后再考虑做事的效率
- 假设有8个核心的处理器，一个可以并行处理的任务，该如何处理？



这里展示的是
理想的任务划分！

任务划分 – 实际情况

- 需要一个好的任务划分策略，不仅让所有核有事可做，还要能高效做事。
 - 负载均衡 – load-balance

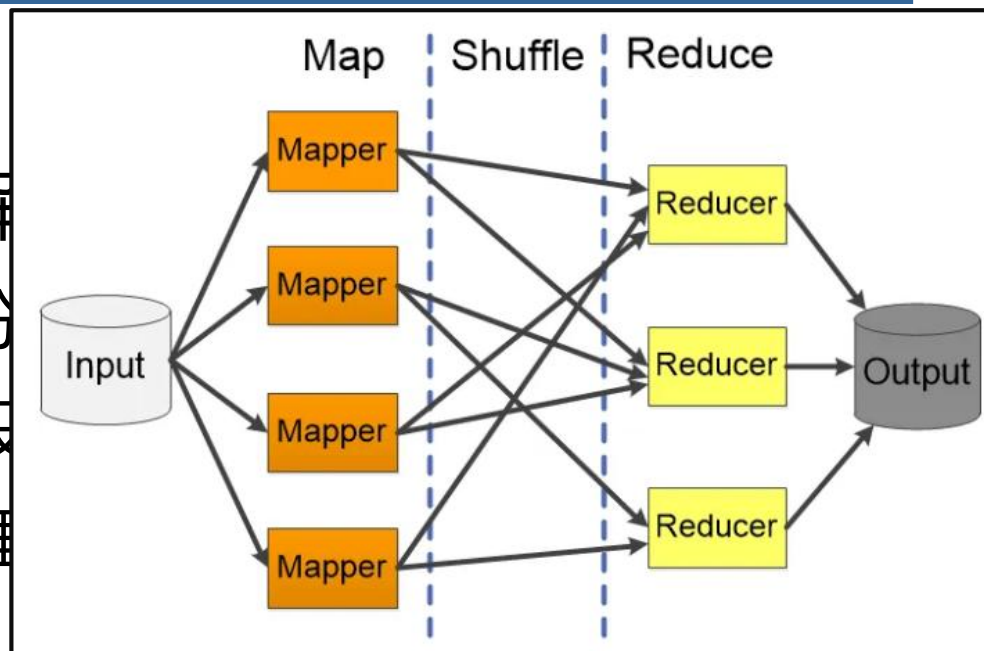


...

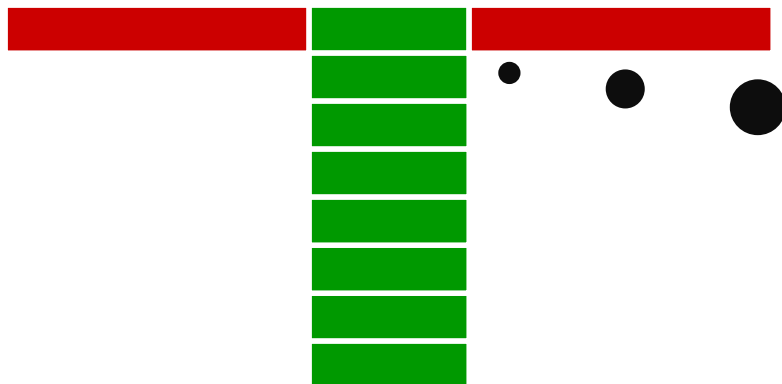
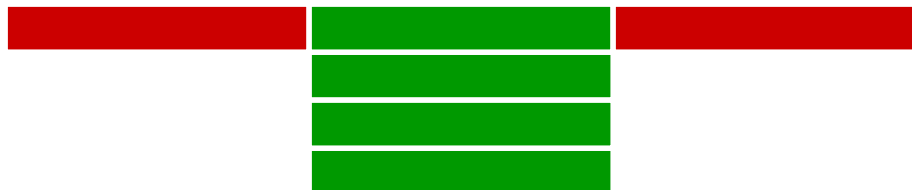
这里展示的是
实际情况！

多核处理器和Amdahl定律

- 任务由: **serial part** 和
- 一个可行的利用多核解
 - step 1: 将输入数据划分
 - step 2: 对每个输入片段
 - step 3: 汇集各部分处理
- 随着核数量的增加...
 - 用于处理每个片段的时间? **goes to zero**
 - 用于处理serial part的时间? **remains the same**
 - 最终, serial part 会 **主导** 任务的整体进度。



多核处理器和Amdahl定律



这里展示的是
现实场景!

多核性能陷阱：Amdahl定律

$$\frac{\text{affected execution time}}{\text{amount of improvement}} + \text{execution time unaffected}$$

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{\text{improvement factor}} + T_{\text{unaffected}}$$

多核性能陷阱：Amdahl定律

- 提升一个计算机系统的某一部分并期望获得系统整体性能的成比例提升，是难以实现的。

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{\text{improvement factor}} + T_{\text{unaffected}}$$

- 示例: multiply 操作占据一个总运行时间100s的程序中的80s;
 - 问: 将multiply的性能提升多少倍可以获得 5倍总体性能提升?

$$20 = 80/n + 20$$

不可能做到!

并行程序设计

- **问题：**我们有了多核处理器之后，应用程序就可以直接提升性能了吗？
- **回答：**并不能，因为软件需要按照并行化的方式去写。对吗？
- 多核处理器面临的困难：
 - 任务划分和负载均衡
 - 协调 和 同步
 - 通信开销
 - 你如何写并行程序？
 - ... 在不知道底层硬件架构细节的情况下？

HW

SW

并行和同步

- 如何写出正确的并程序？
 - 需要理解和掌握下面的核心技术：
 - [Cache coherence](#)
 - [Synchronization](#)
 - [Memory consistency](#)
- 那么，需要知道哪些primitives来设计和实现正确的并程序呢？

In the following...

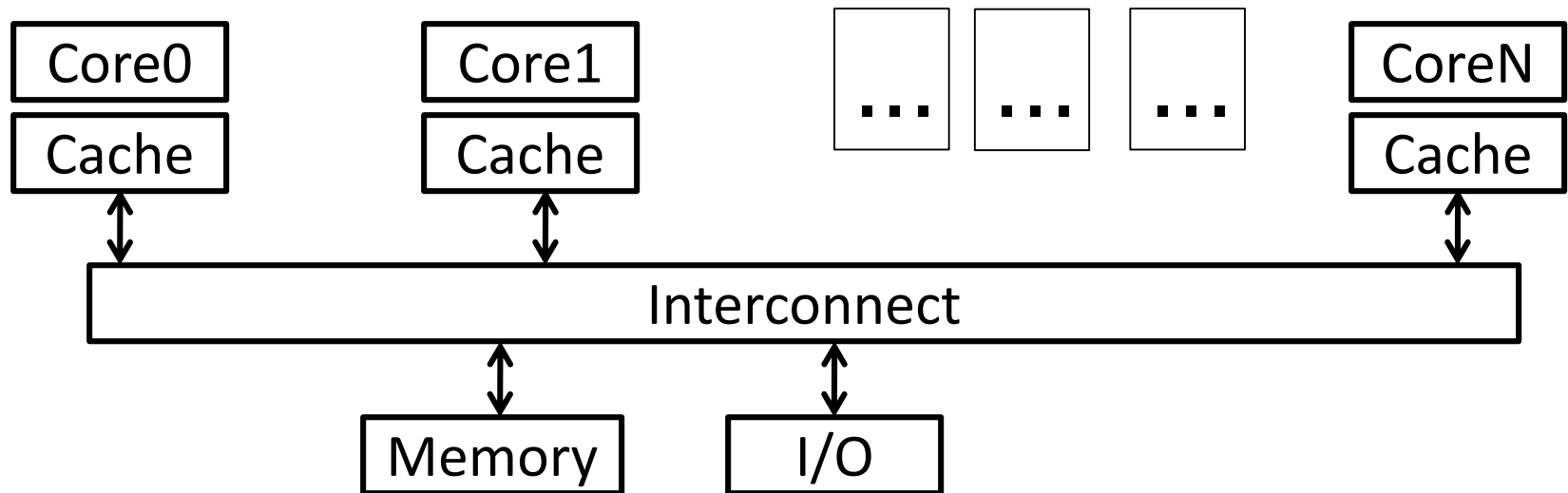
- Understand Cache Coherence problem
- Synchronizing parallel programs
 - Atomic Instructions
 - HW support for synchronization
- How to write parallel programs?
 - Threads and processes
 - Critical sections, race conditions, and mutexes
- We just take you to the very beginning of parallel programming

Parallelism and Synchronization

- **Cache Coherence Problem:** What happens when two or more processors **cache** the *shared* data?
 - We have talked about this issue in our class.
- In other words, the view of memory held by two different processors is through their individual caches.
- Without coherence mechanism, processors can see different (**incoherent**) values to the *same physical* memory location (shared data).

Shared Memory Multiprocessors

- Shared Memory Multiprocessor (SMP)
 - Typical (today): 2 – 8 cores each
 - HW provides ***single physical address space*** for all processors



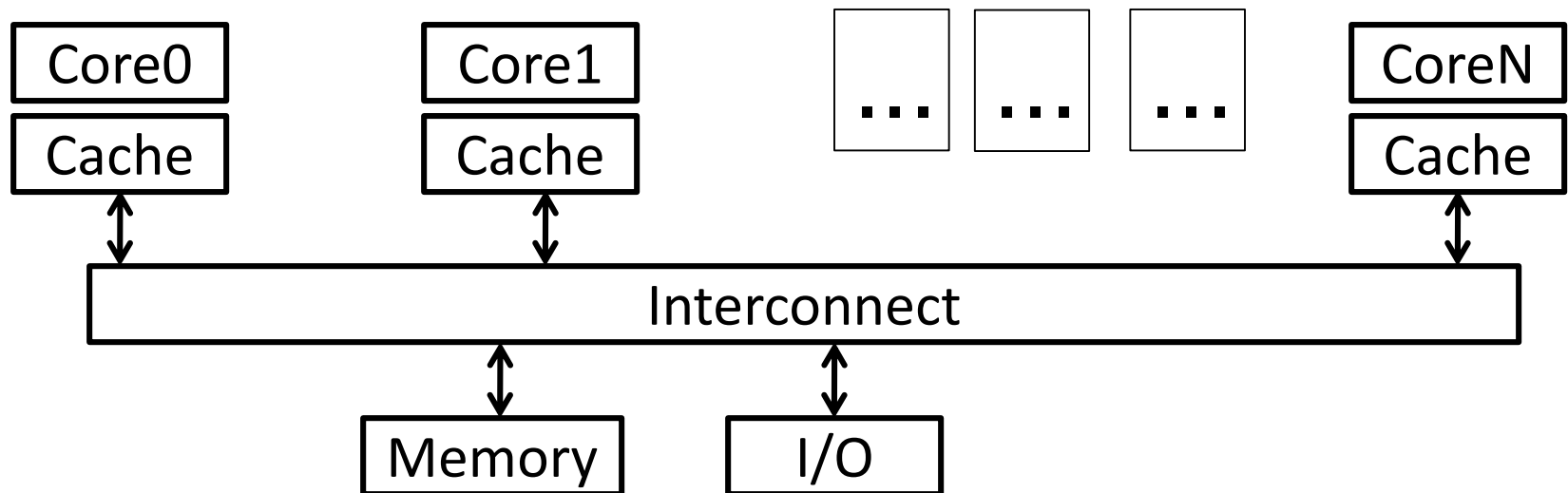
Cache Coherence Problem

- Thread A (on Core0)

```
for(int i = 0; i < 5; i++){  
    x = x + 1;  
}
```
- Thread B (on Core1)

```
for(int j = 0; j < 5; j++){  
    x = x + 1;  
}
```
- What will the value of x be after both loops finish?

Start: x = 0 // x is shared by thread A and B.



Cache Coherence Problem

- Thread A (on Core0)

```
for(int i = 0, i < 5; i++) {  
    x = x + 1;  
}
```

Thread B (on Core1)

```
for(int j = 0; j < 5; j++) {  
    x = x + 1;  
}
```

Atomic operation?

Write what level of memory hierarchy?

Cache Coherence Problem

- Thread A (on Core0)

Thread B (on Core1)

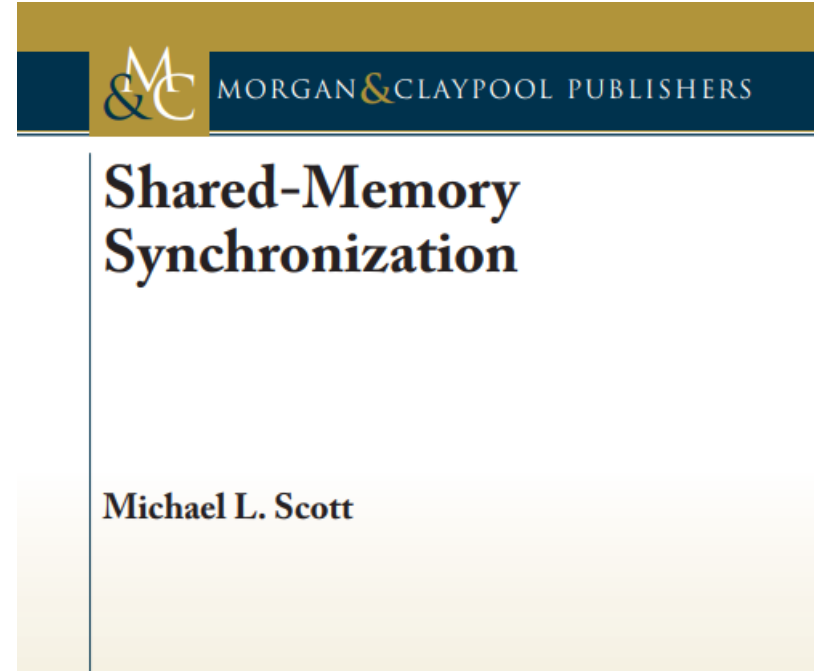
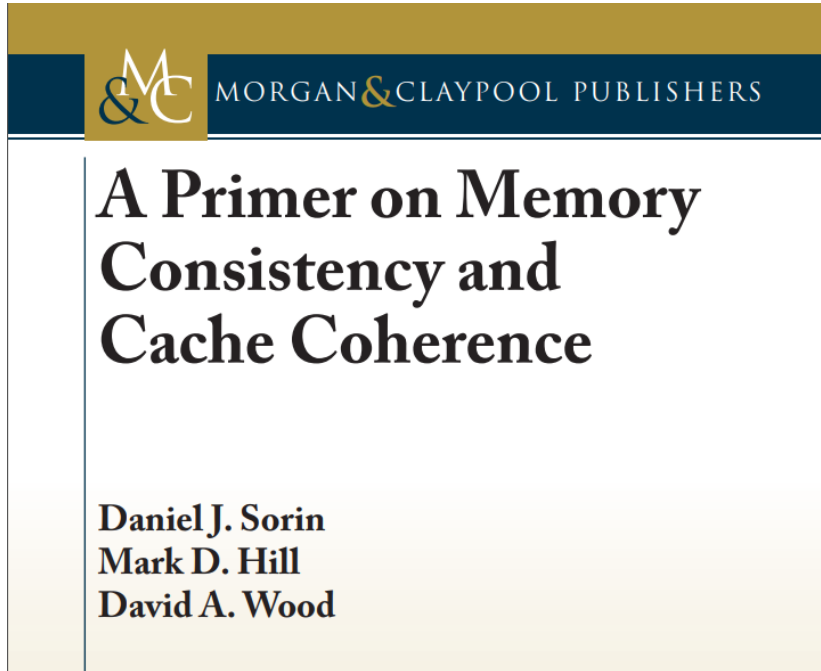
```
for(int i = 0, i < 5; i++) {  
    LW $t0, addr(x)  
    ADDIU $t0, $t0, 1  
    SW $t0, addr(x)  
}
```

```
for(int j = 0; j < 5; j++) {  
    LW $t0, addr(x)  
    ADDIU $t0, $t0, 1  
    SW $t0, addr(x)  
}
```

Cache Coherence Problem

- Thread A (on Core0)
for(int i = 0, i < 5; i++) {
 x = x + 1;
}
- Thread B (on Core1)
for(int j = 0; j < 5; j++)
 x = x + 1;
- What will the value of x be after both loops finish?
 - a) 6
 - b) 8
 - c) 10
 - d) All of the above
 - e) None of the above

Wish to learn more...



- Many topics are not listed and mentioned in our class.
- You have many opportunities to learn if you want to.

结课提醒!!!

- CA is a fundamental and kernel course to CS.
- We just take you to the very beginning of CA.
- I suggest to read, learn, and practice more and more in your future work.
- Keep in mind that:
 - Always write your **course report** by yourself.
 - Please submit your **HW** and **Practice** in time.
 - Prepare your **Final Exam** with your best effort.
- Thanks all.